

IT STUDENT MANAGEMENT SYSTEM

Final Project Closure & Evaluation Report

A Unified Web-Based Platform for Educational Management

Prepared by	Avishek Pandey
Course	IT Project Management
Supervisor	Prof. Deevya Seengal
Methodology	Agile – Scrum Framework
Duration	13 Weeks
Status	Project Closed

Contents

1. Executive Summary	4
2. Project Overview	5
2.1 Scope Delivered	5
2.2 Target Users.....	5
2.3 Technology Stack.....	6
3. Project Execution Summary	7
3.1 Development Approach	7
3.2 Sprint Delivery Summary	7
3.3 Project Timeline Overview	8
4. Deliverables & Outcomes.....	9
5. System Performance & Evaluation	10
5.1 Functional Performance.....	10
5.2 Testing Results.....	10
5.3 Reliability	10
6. Market Alignment & Practical Relevance.....	11
6.1 Identified Market Needs.....	11
6.2 Evaluation Against Market Needs	11
7. Budget & Resource Utilization.....	12
7.1 Cost Summary.....	12
7.2 Cost Efficiency Analysis.....	12
8. Challenges & Resolutions	13
9. Key Achievements	14
10. Lessons Learned.....	15
11. Limitations.....	16
12. Future Improvements	17
13. Project Closure Statement	18
14. Appendices	19
Appendix A: Sprint Summary – Detailed Deliverables	19
Sprint 1 – Authentication & Student Profile Module	19
Sprint 2 – Learning Services Module.....	19
Sprint 3 – Teacher Management Module	19
Sprint 4 – Admin Panel & System Integration.....	19
Appendix B: Risk Register – Final Status	19
Appendix C: Work Breakdown Structure (WBS)	20
Appendix D: System Architecture Overview	20

Appendix E: GitHub Repository	20
15. References.....	22

1. Executive Summary

This section provides a high-level overview of the project's outcomes, methodology, and overall success.

The IT Student Management System was successfully designed, developed, and evaluated as a centralized, web-based platform built to streamline academic and administrative operations within educational institutions. The project was executed over a 13-week development lifecycle using the Agile Scrum methodology, enabling iterative, structured progress and continuous quality refinement at each stage.

The final system integrates four core functional modules into a unified platform: student profile management, learning resource distribution, attendance and grade management, and administrative control. Each module was developed, tested, and integrated across four dedicated development sprints, culminating in a stable and fully documented release.

All primary project objectives were achieved within the defined scope and timeline. The system demonstrated reliable performance across functional, integration, and security testing phases. No critical failures were recorded during validation. The project delivers a practical, cost-effective solution that is directly aligned with the operational needs of educational institutions in emerging markets, where affordability, simplicity, and scalability are paramount priorities.

Duration	Sprints	Modules	Project Status
13 Weeks	4	4	Closed

2. Project Overview

This section outlines the purpose, scope, target users, and technology stack of the IT Student Management System.

The purpose of this project was to address widespread inefficiencies in academic data management across educational institutions that rely on multiple disconnected and often manual systems. Student attendance, grade records, learning materials, and administrative controls were typically managed through separate tools with no unified access or centralized oversight. This fragmentation created data silos, increased administrative workload, and reduced transparency for all stakeholders.

The solution developed is a role-based web application that centralizes all core academic operations into a single secure platform, eliminating redundancy and ensuring consistent, real-time data availability for administrators, teachers, and students.

2.1 Scope Delivered

- User authentication and role-based access control (RBAC)
- Student profile creation, management, and secure data storage
- Learning material upload, class-based filtering, and student access control
- Attendance recording and grade entry by teachers
- Administrative control panel for user and class management
- Integrated database architecture with seamless inter-module data flow

2.2 Target Users

User Role	Primary Responsibilities	Access Level
Administrator	Manage all accounts, assign roles, oversee system	Full System Access
Teacher	Upload materials, record attendance, enter grades	Class-Level Access
Student	View profile, access class resources, review grades	Read-Only Access, File Handling Access
Parents	View Student profile, access student stats, view students' grades	Read-Only access

2.3 Technology Stack

Component	Technology	Purpose
Backend Framework	Django (Python 3.10+)	Application logic, ORM, RBAC, authentication
Database	SQLite 3	Relational data storage for all system entities
Frontend	HTML5 / CSS3	Role-based dashboards and user interface
Version Control	Git / GitHub	Source code management and sprint tracking
Authentication	Django Auth Framework	Session management, CSRF, password hashing
File Storage	Django Media Files	Class-linked resource upload and retrieval

3. Project Execution Summary

This section details the Agile execution approach, sprint outcomes, and overall project timeline.

3.1 Development Approach

The project was executed using Agile Scrum methodology with four structured development sprints, each lasting two weeks. This iterative approach ensured that each module was built, tested, and reviewed before the next sprint began, enabling early detection of issues and continuous integration of stakeholder feedback.

Scrum ceremonies applied throughout the project included Sprint Planning, Sprint Reviews, and Sprint Retrospectives. A prioritized Product Backlog was maintained and refined after each sprint, ensuring the team remained focused on the highest-value deliverables at all times. Daily self-management logs were maintained in lieu of daily stand-ups, given the project's solo development context.

3.2 Sprint Delivery Summary

Sprint	Focus Area	Key Deliverables	Status
Sprint 1	Authentication & Profiles	Login system, admin account, RBAC, student profile module	Completed
Sprint 2	Learning Services	File upload system, class-based filtering, student download access	Completed
Sprint 3	Teacher Management	Teacher dashboard, attendance recording, grade entry, class restrictions	Completed
Sprint 4	Admin Panel & Integration	Admin dashboard, user provisioning, full module integration, integration testing	Completed

3.3 Project Timeline Overview

Phase	Weeks	Key Activities	Status
Planning & Scope Definition	Week 1	Scope definition, stakeholder identification, backlog creation	Completed
System Design	Week 2	Database schema, wireframes, architecture planning	Completed
Sprint 1 Development	Weeks 3–4	Authentication, student profiles, RBAC	Completed
Sprint 2 Development	Weeks 5–6	Learning services, file management	Completed
Sprint 3 Development	Weeks 7–8	Teacher module, attendance, grades	Completed
Sprint 4 Development	Weeks 9–10	Admin panel, full system integration	Completed
Testing & Validation	Weeks 11–12	Functional, integration, security, UAT	Completed
Deployment & Documentation	Week 13	Final deployment, documentation, closure	Completed

4. Deliverables & Outcomes

This section confirms all planned project deliverables and evaluates their completion status.

Deliverable	Description	Status
Functional Web Application	Fully operational IT Student Management System deployed locally	Completed
Student Profile Module	Student data storage, class assignment, attendance and grade views	Completed
Learning Services Module	Teacher upload system with class-based student access control	Completed
Teacher Management Module	Attendance recording, grade entry, and teacher dashboard	Completed
Admin Control Panel	User creation, role assignment, class management, and oversight tools	Completed
Role-Based Dashboards	Distinct, tailored dashboards for Admin, Teacher, and Student roles	Completed
GitHub Repository	Full source code with structured commits aligned to sprints	Completed
Technical Documentation	Installation guide, architecture description, user manual, FAQs	Completed
Test Report	Results for functional, integration, security, and UAT test phases	Completed
Project Closure Report	This document — comprehensive evaluation and formal closure	Completed

All planned deliverables were completed within the defined 13-week timeline. No scope reductions were required, and no deliverables were deferred to a future release. The system is stable, documented, and ready for further enhancement or real-world deployment.

5. System Performance & Evaluation

This section evaluates the functional performance, testing outcomes, and overall reliability of the delivered system.

5.1 Functional Performance

All system modules performed as expected following final integration. Role-based access control was correctly enforced across all user types, ensuring that each role could only access features and data within its defined permission boundary. Data flow between modules, for example, teacher-uploaded resources appearing in the student portal, and teacher-recorded attendance updating student profiles, operated seamlessly without manual intervention.

The system correctly handled all primary user workflows: student login and profile access, teacher material upload and academic data entry, and administrator user and class management. No functional defects were identified during the final testing phase.

5.2 Testing Results

Test Type	Outcome	Details
Functional Testing	Passed	All 100% of user stories and acceptance criteria verified successfully.
Integration Testing	Seamless	Inter-module data flow confirmed across all four module combinations.
Security Testing	Prevented	All unauthorized access attempts blocked; URL manipulation validation confirmed.
User Acceptance Testing	Positive	Peer testers rated interface clarity, navigation, and usability positively.
Regression Testing	Passed	No previously passing tests broken by Sprint 4 integration changes.

5.3 Reliability

The system demonstrated stable and reliable performance under all simulated normal usage conditions during the testing phases. No critical system failures, data corruption incidents, or authentication breakdowns were observed. Error handling was implemented consistently across all modules, with user-friendly error messages replacing technical error outputs for all validation failures.

Performance under concurrent test scenarios (simulating multiple users accessing the system simultaneously via separate browser sessions) produced no observable degradation in response

time or data integrity. The system is considered production-ready for small-to-medium institutional deployments.

6. Market Alignment & Practical Relevance

This section evaluates how well the delivered system addresses real-world market needs in its target regions.

6.1 Identified Market Needs

The system was designed for educational institutions in Nepal, India, Bangladesh, and Pakistan, emerging markets with large student populations that are actively transitioning toward digital academic management. Research into these markets identified the following primary requirements that institutions in these regions consistently seek:

- Centralized academic management to replace fragmented, manual record-keeping.
- Affordable, cost-effective solutions accessible to institutions with limited IT budgets.
- User-friendly interfaces suitable for users with varying levels of technical literacy.
- Scalable architecture capable of growing with institutional enrolment.
- Secure, role-based access to protect student data and maintain governance.

6.2 Evaluation Against Market Needs

Market Requirement	Achieved	Evaluation Notes
Centralised Management	Yes	All four modules integrated into a single platform with unified login.
Cost Efficiency	Yes	Built entirely on open-source technologies; zero licensing cost.
Ease of Use	Yes	Role-based UI ensures each user sees only relevant features.
Scalability	Partial	SQLite limits high-volume deployments; PostgreSQL migration planned.
Data Security	Yes	RBAC, session management, and backend validation fully implemented.
Accessibility	Partial	Desktop-optimised; mobile-responsive design deferred to next version.

The system strongly aligns with the operational needs of educational institutions in emerging markets. Areas marked as Partial — specifically scalability and mobile responsiveness — represent planned enhancements rather than functional deficiencies, and do not inhibit deployment in small-to-medium institution environments.

7. Budget & Resource Utilization

This section summarizes the resources consumed and the overall cost efficiency of the project.

7.1 Cost Summary

Resource / Expense	Estimated Cost (AUD)	Category
Development Laptop / PC (amortized hardware cost)	\$1,200	Technical
Internet Connectivity & Utilities	\$100	Operational
Software Tools (Django, SQLite, Git, VS Code)	\$0	Technical
Design & Prototyping Tools (Wireframing)	\$100	Design
Testing (Peer-based UAT)	\$0	Quality Assurance
Miscellaneous (Printing, Documentation)	\$100	Operational
TOTAL ESTIMATED PROJECT COST	~\$1,600 AUD	

7.2 Cost Efficiency Analysis

The project achieved exceptional cost efficiency by leveraging exclusively open-source technologies. The Django web framework, SQLite database, Git version control, and Visual Studio Code editor all carry zero licensing costs, dramatically reducing the total expenditure relative to equivalent proprietary solutions.

The primary cost contributor was the amortized hardware cost of the development machine. In a commercial project context, this cost would be spread across multiple projects, further reducing the effective per-project investment. The total estimated project cost of approximately AUD \$1,400 demonstrates that a fully functional, enterprise-quality student management platform can be delivered at a fraction of the cost of commercial alternatives — a critical advantage for the target markets of Nepal, India, Bangladesh, and Pakistan, where IT budgets in educational institutions are typically limited.

8. Challenges & Resolutions

This section documents the key challenges encountered during the project and the strategies employed to overcome them.

Challenge 1: Full Lifecycle Management as a Solo Developer

Challenge: Managing all project roles simultaneously, project manager, developer, system designer, tester, and documentation author, created significant workload pressure, particularly during sprint transitions when planning and development tasks overlapped.

Resolution: A disciplined sprint planning approach was adopted from the outset, with detailed task lists and time-boxed daily goals. Strict backlog prioritization ensured that Minimum Viable Product features were protected at all times. When time pressure was identified, non-critical enhancements were deferred to the backlog without affecting core deliverables.

Challenge 2: Implementing Scalable Role-Based Access Control

Challenge: Ensuring that permission logic was both secure and maintainable across four distinct user roles presented a significant design challenge. Early implementations embedded access checks at the view level, leading to duplication and inconsistency risks.

Resolution: Permission logic was centralized using Django's built-in decorators and custom middleware. This approach ensured that access control was enforced consistently at every application route, eliminating duplication and making the permission system easier to audit and extend.

Challenge 3: Cross-Module System Integration

Challenge: Ensuring accurate and real-time data flow between all four modules, for example, teacher-uploaded resources appearing instantly in student portals, or attendance data updating student profiles, posed complex data consistency challenges during the integration sprint.

Resolution: A unified database schema with carefully designed foreign key relationships was established during the system design phase. Integration testing was prioritized at the start of Sprint 4, and dedicated integration test cases were written for each cross-module data flow before the final release was approved.

9. Key Achievements

This section highlights the most significant successes delivered by the project.

1	Successful end-to-end development of a multi-module web application within a 13-week semester. The four-module system was built, integrated, tested, and documented on schedule, demonstrating disciplined project execution.
2	Secure implementation of Role-Based Access Control across all user types. RBAC was enforced at the middleware and view level, with backend validation preventing unauthorized access via URL manipulation.
3	Full Agile Scrum lifecycle execution from initiation through to formal project closure. Sprint planning, reviews, retrospectives, and backlog management were applied throughout, producing measurable improvements with each iteration.
4	Delivery of a real-world applicable solution aligned with emerging market needs. The system is directly deployable in educational institutions across Nepal, India, Bangladesh, and Pakistan with minimal configuration.
5	High system stability and positive usability outcomes. No critical failures in testing; positive peer UAT feedback confirmed that the role-based UI significantly reduced interface complexity.
6	Cost-effective development using exclusively open-source technologies. Total estimated cost of ~AUD \$1,400 with zero software licensing expenditure, demonstrating viability for low-budget institutions.

10. Lessons Learned

This section reflects on the key insights gained throughout the project lifecycle.

Lesson	Application to Future Projects
Incremental development significantly reduces project risk.	Future projects will continue to use sprint-based delivery to ensure early and continuous testing rather than end-stage integration.
Early and continuous testing prevents costly integration failures.	Integration test cases will be written for each module before it is considered complete, not deferred to a final phase.
Continuous stakeholder feedback improves both usability and design.	Sprint reviews will include structured feedback sessions with user representatives, not only technical reviews.
Project planning must remain adaptive and not treated as a one-time exercise.	Backlog refinement will be a standing agenda item at every sprint review, with risks reassessed at each iteration.
Documentation should be developed concurrently with code.	Technical documentation will be updated at the end of each sprint rather than completed as a final stage activity.

11. Limitations

This section transparently identifies the current boundaries and constraints of the delivered system.

Limitation	Impact	Severity
SQLite database limits concurrent high-volume access	Performance may degrade under heavy simultaneous usage in large institutions	Medium
No cloud deployment in current release	System access is restricted to local network; remote access requires additional setup	Medium
No mobile-responsive interface	Usability reduced on smartphones, which are primary devices in target markets	Medium
No real-time notification system	Users must manually check for updates; no push alerts for uploads or grade changes	Low
No public self-registration	All accounts must be created by Admin, creating a provisioning bottleneck at scale	Low
No analytics dashboard	Administrators cannot view aggregated performance trends or usage statistics	Low

12. Future Improvements

This section outlines the planned enhancements prioritised for future development iterations.

Priority	Enhancement	Description	Addresses
1	PostgreSQL Migration	Replace SQLite with PostgreSQL for concurrent high-volume access	Scalability
2	Cloud Deployment	Deploy to AWS or Heroku for remote access and institutional hosting	Accessibility
3	Mobile-Responsive UI	Implement responsive CSS framework for full smartphone compatibility	Usability
4	Notification System	Email and in-app alerts for resource uploads, grade updates, deadlines	Engagement
5	Analytics Dashboard	Performance trends, attendance patterns, and usage insights for Admin	Oversight
6	REST API Layer	Expose API endpoints for integration with third-party LMS and mobile apps	Interoperability
7	Multi-Language Support	Localisation for Nepali, Hindi, Bengali, and Urdu	Market Reach

13. Project Closure Statement

FORMAL PROJECT CLOSURE

The IT Student Management System project has been successfully completed within the defined scope, timeline, and objectives. All critical functionalities have been implemented, tested, and validated. All planned deliverables have been produced and reviewed. No major outstanding defects or unresolved issues remain.

The system is stable, functional, fully documented, and ready for further enhancement or deployment in a real-world institutional environment. The Agile Scrum methodology was applied successfully throughout the project, ensuring structured delivery, continuous improvement, and stakeholder alignment at every stage.

This project is hereby formally closed. All project artefacts, documentation, and source code have been archived in the GitHub repository and are available for future reference, enhancement, and deployment.

Project Status: CLOSED ✓

14. Appendices

Appendix A: Sprint Summary – Detailed Deliverables

Sprint 1 – Authentication & Student Profile Module

- Implemented Django authentication system with secure login, logout, and session management.
- Created company-provisioned Admin superuser account with elevated access.
- Developed role-based dashboard redirection: Admin, Teacher, and Student routes.
- Built Student Profile model with fields for name, student ID, class, contact, and academic summary.
- Implemented class assignment system linking students to class entities.
- Conducted functional testing on all authentication flows.

Sprint 2 – Learning Services Module

- Developed file upload interface allowing teachers to submit academic materials.
- Implemented class-ID-based resource linking and folder structure for file organisation.
- Built class-based content filtering ensuring students access only enrolled class materials.
- Implemented backend URL manipulation prevention for unauthorized resource access.
- Conducted upload, retrieval, and access control testing.

Sprint 3 – Teacher Management Module

- Developed the Teacher dashboard with class management tools.
- Implemented daily attendance recording with student-class linkage.
- Built grade entry system for assessment results, linked to student profiles.
- Enforced class-based access restrictions: teachers cannot access unassigned classes.
- Conducted role-based access restriction testing.

Sprint 4 – Admin Panel & System Integration

- Finalised the Administrator dashboard with full user management tools.
- Implemented user account creation, role assignment, and password reset features.
- Integrated all four modules into the unified platform with shared database and login system.
- Conducted comprehensive integration testing across all module data flows.
- Applied UI refinements based on peer UAT feedback.

Appendix B: Risk Register – Final Status

Risk	Likelihood	Impact	Mitigation Applied	Final Status
Scope Expansion	High	Project delay	Strict backlog prioritisation maintained throughout	Mitigated
Time Limitation (Solo)	High	Incomplete features	MVP-first approach; non-critical features deferred	Mitigated
Data Security Vulnerability	Medium	Unauthorized access	RBAC + backend validation + Django security features	Mitigated
File Storage Issues	Medium	Performance impact	File size limits applied; efficient directory structure	Mitigated
Integration Failures	High	System instability	Independent module testing before integration sprint	Mitigated

Appendix C: Work Breakdown Structure (WBS)

1. Project Planning and Documentation
2. Database Modelling and Schema Design
3. Authentication System Development
4. Role-Based Access Control Implementation
5. Student Profile Module Development
6. Learning Resource Upload and Class-Based Filtering
7. Teacher Attendance and Grade Management Module
8. Admin Account Provisioning and Control Panel
9. System Integration and Comprehensive Testing
10. Final Documentation, Deployment, and Project Closure

Appendix D: System Architecture Overview

The system follows Django's Model-View-Template (MVT) architectural pattern, with four distinct Django applications sharing a central SQLite database. The database schema includes the following primary entities:

- User — Stores credentials, role assignments, and session data.
- StudentProfile — Student-specific data, class assignment, and academic summary.
- Class — Defines class entities from Level 1 through Masters.
- Resource — Academic materials with class-ID foreign key linkage.
- Attendance — Daily records linked to student, class, and teacher.
- Grade — Assessment results linked to student, class, and assessment type.

Appendix E: GitHub Repository

Repository URL: <https://github.com/avishekpandey/it-student-management>

All commits follow a structured format aligned to sprint milestones: [SprintN] – Description of change. The repository includes the full source code, requirements.txt, README.md, and the /docs/ directory containing technical documentation and wireframes.

15. References

Beck, K., et al. (2001). Manifesto for Agile Software Development. Retrieved from <https://agilemanifesto.org>

Django Software Foundation. (2024). Django Documentation (Version 4.2). Retrieved from <https://docs.djangoproject.com>

Project Management Institute. (2021). A Guide to the Project Management Body of Knowledge (PMBOK Guide) (7th ed.). PMI.

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. Scrum.org. Retrieved from <https://scrumguides.org>

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

UNESCO. (2023). Digital Transformation in Education: Emerging Markets Report. United Nations Educational, Scientific and Cultural Organization.

OWASP Foundation. (2023). OWASP Top Ten Web Application Security Risks. Retrieved from <https://owasp.org/www-project-top-ten/>